Instructivo de Práctica — Sesión 6: Gastos del viaje desde el backend

> 📌 **Qué vas a entregar:** al terminar esta práctica tendrás **(1)** el backend de control de gastos del módulo anterior corriendo en tu computador y **(2)** ExploraEC con una sección nueva, **Gastos**, que se registra e inicia sesión contra ese backend y lista los gastos de tu viaje (por ejemplo, «Almuerzo en el Mercado Central — 6.50, comida») con los tres estados de la Sesión 3 (cargando, vacío, error). Los lugares y el mapa de las Sesiones 1-5 se conservan tal cual. Como en las demás sesiones, lo que se sube es **tu práctica** (en clase se indica cómo y cuándo).

Objetivo

Levantar localmente un backend REST real (el del módulo anterior) y hacer el primer consumo de una API desde ExploraEC: encadenar con `async` / `await` un flujo de varias llamadas HTTP (registro → inicio de sesión → listar gastos), integrarlo en un `GastosController` (GetX, patrón de la Sesión 4), reutilizar la interfaz de estados (loading/vacío/error) de la Sesión 3 y traducir a mensajes legibles los errores de red y los errores HTTP más comunes (sin conexión, tiempo agotado, 401, 400, 422, 5xx).

Requisitos previos

- Haber completado las prácticas de las Sesiones 4 y 5 (`PlacesController` con `Obx` , y mapa real con permisos y posición del usuario funcionando).
- Una carpeta de trabajo común `exploraec-curso` con la app (`exploraec`) y el backend (`proyecto-curso-spec-kit`) lado a lado; se prepara en el Paso 2.
- **Docker** y **git** instalados:
- Docker Desktop (Windows/Mac) o Docker Engine con el plugin Compose (Linux). Comprobar: `docker --version` y `docker compose version` .
- En Windows, Docker Desktop requiere WSL2 activado.
- `git --version` .
- Unos 2 GB libres de disco y conexión a internet (la primera construcción descarga imágenes de Docker).

- **Recomendado: haber hecho los Pasos 2 y 3 antes de la sesión.** La primera construcción de Docker puede tardar varios minutos; en clase, con todo el grupo descargando a la vez, tarda más.
- Si no puedes correr Docker en tu equipo, el docente te indicará la dirección de su backend en la red del aula (Paso 3, Opción B).

Tiempo estimado

Total: 65 minutos

Paso	Descripción	Tiempo
0	Recap de la Sesión 5: ExploraEC todavía no habla con ningún servidor	5 min
1	Qué es el backend de gastos (contexto del módulo 2)	4 min
2	Traer el código del backend y configurar `.env`	4 min
3	Levantar el backend con Docker	8 min
4	Verificar el backend: `/health`, `/docs` y registro/login con `curl`	8 min
5	Traer el código de la Sesión 6 y ejecutar ExploraEC	5 min
6	Revisar `GastosApiService`	7 min
7	Conectar `GastosController`: registro → login → listar gastos	10 min
8	Probar los 3 estados y los errores con el servidor real	8 min
9	Aplicar la teoría: un timeout explícito	6 min
10 *(opcional)*	Deslizar para actualizar — no cuenta dentro de los 65 min	10 min

Paso 0 — Recap de la Sesión 5: ExploraEC todavía no habla con ningún servidor (5 min)

Por qué se hace

Antes de traer código nuevo conviene ver el límite que hoy se empieza a cruzar: Inicio y Mapa ya comparten el `PlacesController` (Sesión 4) y el mapa usa tu posición real (Sesión 5), pero **todo lo que muestra la app está escrito en el código** (`lugaresEjemplo` , los mismos 6 sin importar dónde estés) y se pierde al cerrar. Hoy el turista empieza a registrar **lo que gasta** durante el viaje, y eso sí viene de un servidor. Este paso se hace **con tu proyecto tal como quedó en la Sesión 5**, antes de traer la rama de hoy.

Instrucciones

1. En tu proyecto `exploraec` (si ya hiciste el Paso 2, está en `exploraec-curso/exploraec` : la **terminal B**), confirmar que el trabajo de la Sesión 5 está guardado:

```
cd exploraec-curso/exploraec   # solo si aún no estás dentro de tu proyecto
git status
```

Si aparecen cambios sin guardar, hacer `git add -A && git commit -m "Sesión 5"` (el Paso 5 de abajo sobrescribe `lib/`).

2. Ejecutar la app con el emulador ubicado en Quito y repasar lo construido. En la terminal B:

```
flutter emulators
flutter emulators --launch <id-del-emulador>
flutter devices
adb emu geo fix -78.5125 -0.2201
flutter run
```

`flutter emulators` lista los emuladores (la primera columna es el `<id-del-emulador>` , por ejemplo `Pixel_10_Pro`); `flutter devices` debe mostrarlo ya encendido (por ejemplo `emulator-5554`); `adb emu geo fix` fija la ubicación simulada en Quito (**primero la longitud y luego la latitud**; `adb` viene en la carpeta `platform-tools` del SDK de Android; si tu terminal no lo encuentra, usa *Extended controls* → *Location* del emulador con latitud -0.2201 y longitud -78.5125). Verás Inicio con `ExploraEC (6)` ; toca **Mapa** (acepta el diálogo de permiso de ubicación si aparece): el mapa se centra en Quito con los marcadores.

3. Cambiar la ubicación simulada a otra ciudad y volver a abrir el Mapa. Sin detener la app,

en otra terminal:

```
adb emu geo fix -79.8891 -2.1894
```

(Guayaquil.) En la app toca **Inicio** y luego **Mapa** otra vez: los 6 marcadores siguen en Quito. Los lugares **se quedan así** en esta serie de prácticas; lo que hoy se conecta a una API es otra cosa. Cuando termines, detén la app con `q` en la terminal donde corre `flutter run`. 4. Responder en una frase: si el turista anota un gasto («Almuerzo, 6.50»), ¿dónde debería guardarse para no perderlo al cerrar la app, y qué tendría que pasar entre el teléfono y ese lugar? (Pista: una llamada de red tarda, y puede fallar.)

Resultado esperado

Se puede explicar que los datos actuales son fijos y locales, y que hablar con un servidor introduce dos cosas nuevas: **espera** (hay que usar `Future` / `async` / `await`) y **fallos** (hay que mostrar estados de carga y de error, como en la Sesión 3).

Errores comunes

- `git status` **muestra cambios de la Sesión 5 sin guardar**. *Solución:* guardarlos con un commit antes de seguir: el Paso 5 sobrescribe `lib/`.
- **El Mapa muestra el mensaje de ubicación desactivada**. *Solución:* repetir el Paso 3 de la Sesión 5 (fijar la ubicación simulada y aceptar los diálogos de Google).

✅ Antes de continuar, confirma que...

- ☐ El trabajo de la Sesión 5 está guardado en un commit.
- ☐ Viste que los 6 marcadores no cambian al mover la ubicación simulada.
- ☐ Puedes decir por qué una llamada a un servidor obliga a pensar en asincronía y en errores.

Paso 1 — Qué es el backend de gastos (contexto del módulo 2) (4 min)

> Este paso es de lectura. Si ya trabajaste con este backend en el módulo 2, repásalo en un minuto; si **no** estuviste en ese módulo, es lo único que necesitas saber para seguir.

Por qué se hace

Una app móvil no guarda por sí sola los datos de un usuario de forma compartida y duradera: eso lo hace un **backend**, un programa que corre en un servidor y que la app consulta por internet. En el módulo anterior se construyó uno: una **API de control de gastos personales**. A partir de hoy ExploraEC lo usa para una sección nueva, **«Gastos del viaje»** (lo que el turista gasta mientras explora). No necesitas modificarlo ni entender su código interno: se usa como una caja negra con un contrato.

Qué es y qué contiene

- **API REST** hecha con **FastAPI** (Python). Habla **JSON** sobre HTTP y se documenta sola en `/docs` (Swagger).
- **Base de datos PostgreSQL**, que guarda usuarios y gastos.
- **Autenticación con JWT**: el usuario se registra, inicia sesión y recibe un *token*; en cada petición siguiente lo envía en la cabecera `Authorization: Bearer <token>`. El token **caduca a los 30 minutos**.
- **Docker Compose** lo arma todo con un solo comando: tres servicios que arrancan en orden.

Servicio	Qué hace
`db`	La base de datos PostgreSQL 16 donde se guardan usuarios y gastos.
`migrate`	Corre **una sola vez** y termina: crea las tablas (migraciones con Alembic).
`backend`	La API en sí, escuchando en el puerto **8000**. Arranca solo cuando `migrate` terminó bien.

Contrato mínimo que usarás hoy

Método y ruta	Para qué	Cómo se envía
`GET /health`	Comprobar que el servidor y la base de datos responden	—
`POST /usuarios/`	Registrar un usuario	JSON `{"email","password"}` (contraseña de **8 a 72** caracteres)
`POST`	Iniciar sesión y obtener	**Formulario** (`application/x-www-form-urlencoded`)

Método y ruta	Para qué	Cómo se envía
`/usuarios/token`	el token	con `username` (tu correo) y `password`
`GET /gastos/`	Listar **tus** gastos (cada usuario ve solo los suyos)	Cabecera `Authorization: Bearer <token>`
`POST /gastos/`	Crear un gasto	JSON `{"descripcion","monto","categoria"}` con `categoria` ∈ `comida`, `transporte`, `entretenimiento`, `otros`

Hay reglas de negocio que verás más adelante: un gasto no puede superar **500** acumulados por categoría (responde `400`) y los errores llegan como `{"detail": "..."}` en español.

> Repositorio del backend: https://github.com/cj-murillo/proyecto-curso-spec-kit.

Resultado esperado

Puedes explicar con tus palabras: qué es el backend, que corre **aparte** de la app (en tu computador, en un contenedor Docker) y que la app solo lo alcanza por una dirección y un puerto.

Errores comunes

- Ninguno específico: este paso es de lectura.

✅ Antes de continuar, confirma que...

- ☐ Sabes qué tres servicios arma Docker Compose y cuál es el puerto de la API (8000).
- ☐ Sabes que el inicio de sesión se envía como formulario y no como JSON, y que el token caduca a los 30 minutos.

Paso 2 — Traer el código del backend y configurar `.env` (4 min)

Por qué se hace

El backend es un repositorio aparte del de ExploraEC. Hay que traerlo y, antes de arrancarlo, darle un secreto propio: `SECRET_KEY` es la clave con la que el servidor **firma** los tokens JWT. Sin ella, el backend no arranca. Es un secreto de tu entorno: **nunca se sube al repositorio** (por eso `.env` está en `.gitignore`).

Instrucciones

Ejecuta los sub-pasos **2.1 a 2.6 en orden**; cada bloque de comandos se copia tal cual. Al terminar tendrás esta estructura, con el front y el back lado a lado y abiertos juntos en el IDE:

```
exploraec-curso/
├── exploraec/                  <- la app Flutter (tu proyecto de las Sesiones 1-5)
└── proyecto-curso-spec-kit/    <- el backend de gastos (se clona en el sub-paso 2.2)
```

`exploraec` y `proyecto-curso-spec-kit` siguen siendo **dos repositorios git independientes**; `exploraec-curso` es solo la carpeta que los contiene (no es un repositorio ni se sube a ningún lado).

2.1 — Crear la carpeta común y colocar tu app dentro

Tu app `exploraec` es el proyecto que creaste con `flutter create` en la Sesión 1. Cierra antes el IDE y la app si los tienes abiertos y ejecuta (si tu proyecto no está en `Documentos`, cambia la primera ruta):

```
cd ~/Documentos
mkdir exploraec-curso
mv exploraec exploraec-curso/exploraec
cd exploraec-curso/exploraec
flutter pub get
cd ..
pwd
ls
```

(En Windows PowerShell: `cd $HOME\Documents`, `mkdir exploraec-curso`, `Move-Item exploraec`

`exploraec-curso\exploraec`, `cd exploraec-curso\exploraec`, `flutter pub get`, `cd ..`, `pwd`, `dir`.) `pwd` debe terminar en `exploraec-curso` y `ls` debe mostrar solo `exploraec`. Mover la carpeta no toca tu historial de git ni tus remotos (`origin`, `starter`).

2.2 — Clonar el backend (al lado de `exploraec`, no dentro)

Estando en la carpeta común (`pwd` termina en `exploraec-curso`), clona el repositorio del backend:

```
git clone https://github.com/cj-murillo/proyecto-curso-spec-kit
ls
```

Debe aparecer la carpeta nueva `proyecto-curso-spec-kit` junto a `exploraec`. (En Windows PowerShell usa `dir` en lugar de `ls`.)

2.3 — Entrar al backend y crear tu archivo `.env`

```
cd proyecto-curso-spec-kit
cp .env.example .env
```

(En Windows PowerShell: `copy .env.example .env`.)

2.4 — Generar la `SECRET_KEY` y escribirla en `.env` (un solo comando)

`SECRET_KEY` es la clave con la que el servidor firma los tokens. Este comando genera 64 caracteres aleatorios y los escribe después de `SECRET_KEY=` en tu `.env` (sin comillas ni espacios); funciona igual en Linux, Mac y Windows si tienes Python:

```
python3 -c "import re,secrets,pathlib; p=pathlib.Path('.env');
p.write_text(re.sub(r'(?m)^SECRET_KEY=.*', 'SECRET_KEY='+secrets.token_hex(32),
p.read_text()))"
```

(En Windows puede ser `python` o `py` en lugar de `python3`.) Comprueba el resultado:

```
grep SECRET_KEY .env
```

(En PowerShell: `Select-String SECRET_KEY .env`.) Debe mostrar `SECRET_KEY=` seguido de 64

caracteres, no una línea vacía.

El archivo `.env` queda así (solo cambió la línea de `SECRET_KEY`; el resto, incluido el comentario de la primera línea, se queda como está):

```
SECRET_KEY=9f3a...c1d7
DATABASE_URL=sqlite:///./gastos.db
ACCESS_TOKEN_EXPIRE_MINUTES=30
LOG_LEVEL=INFO
MCP_DEMO_EMAIL=demo@curso.com
MCP_DEMO_PASSWORD=demo1234
MCP_ISSUER_URL=http://127.0.0.1:8000
MCP_RESOURCE_URL=http://127.0.0.1:8000/mcp
```

> `DATABASE_URL` de `.env` **no se usa con Docker**: `docker-compose.yml` la reemplaza por la de PostgreSQL. No la cambies. (Las líneas `MCP_*` se usan en la Sesión 9.)

2.5 — Abrir front y back juntos en el IDE

Vuelve a la carpeta común y ábrela completa:

```
cd ..
code .
```

- **VS Code** (`code .`): abre `exploraec-curso` con las dos carpetas en el explorador lateral. Si el comando `code` no existe, abre VS Code y usa *File → Open Folder...* sobre `exploraec-curso`. Pide que confíes en la carpeta (*Trust*) y, si te sugiere instalar las extensiones de Flutter/Dart, acepta.
- **Android Studio:** *File → Open...* sobre la carpeta `exploraec-curso/exploraec` (abre un proyecto Flutter por ventana; el backend no necesitas editarlo, solo ejecutarlo). Si quieres ver también el backend, ábrelo en una segunda ventana o con VS Code.

2.6 — Preparar las dos terminales que usarás toda la práctica

En VS Code se abren con *Terminal → New Terminal* (dos veces); en cada una entra a su carpeta:

Terminal	Carpeta	Para qué
A — backend	`exploraec-curso/proyecto-curso-spec-kit`	`docker compose ...` y `curl` (Pasos 3, 4 y 8)
B — app	`exploraec-curso/exploraec`	`git`, `flutter pub get`, `flutter run` (Pasos 0, 5, 7, 9 y 10)

```
# Terminal A
cd exploraec-curso/proyecto-curso-spec-kit
# Terminal B
cd exploraec-curso/exploraec
```

(Ajusta el inicio de la ruta a donde hayas creado `exploraec-curso`; dentro de VS Code las terminales ya nacen en la carpeta que abriste.)

Resultado esperado

La carpeta `exploraec-curso` contiene las dos carpetas y cada una es un repositorio distinto. Comprobación rápida, ejecutada desde `exploraec-curso`:

```
ls
git -C exploraec remote -v
git -C proyecto-curso-spec-kit remote -v
```

Debe listar `exploraec` y `proyecto-curso-spec-kit`; el primer `git` muestra tus remotos (`origin` y `starter`) y el segundo `origin https://github.com/cj-murillo/proyecto-curso-spec-kit`.

Además, existe el archivo `.env` en la raíz de `proyecto-curso-spec-kit` con `SECRET_KEY` completada. Comprobación rápida, también desde `exploraec-curso` (debe mostrar tu clave, no una línea vacía):

```
grep SECRET_KEY proyecto-curso-spec-kit/.env
```

(En PowerShell: `Select-String SECRET_KEY proyecto-curso-spec-kit\.env`.)

Errores comunes

- **`git: command not found`** / **`'git' no se reconoce`**. *Solución:* instalar git y abrir una

terminal nueva.

- `fatal: destination path 'proyecto-curso-spec-kit' already exists`. *Solución:* ya lo clonaste antes: entra a la carpeta (`cd proyecto-curso-spec-kit`) y, si quieres lo último, `git pull`.
- `.env` **no aparece al listar la carpeta**. *Solución:* los archivos que empiezan con punto están ocultos: `ls -a` (Linux/Mac) o «mostrar archivos ocultos» (Windows).
- **Se dejó** `SECRET_KEY=` **vacía o con comillas**. *Solución:* sin clave real, los tokens no son seguros o la firma falla. Pega la clave sin comillas.
- `mv: no se puede efectuar stat de 'exploraec'` / `cannot stat 'exploraec'`. *Solución:* no estás en la carpeta que contiene tu proyecto. Entra a ella (`cd ~/Documentos` o donde lo creaste) y repite el `mv`.
- **El IDE sigue mostrando el proyecto en la ruta antigua o marca errores de rutas tras mover la carpeta**. *Solución:* cierra el IDE, ábrelo de nuevo desde `exploraec-curso` y ejecuta `flutter pub get` dentro de `exploraec`.
- **Se clonó el backend *dentro* de** `exploraec`. *Solución:* bórralo (`rm -rf exploraec/proyecto-curso-spec-kit`) y clónalo de nuevo desde `exploraec-curso`. Dentro de tu proyecto aparecería como carpeta sin seguimiento en `git status`, y un `git add -A` la registraría como «repositorio embebido» (un submódulo roto) en tu commit.

✅ Antes de continuar, confirma que...

- ☐ Existe `exploraec-curso/` con `exploraec/` y `proyecto-curso-spec-kit/` dentro, y los abres juntos en el IDE.
- ☐ `.env` existe y `SECRET_KEY` tiene una clave de 64 caracteres.

Paso 3 — Levantar el backend con Docker (8 min)

Por qué se hace

`docker compose up --build` construye la imagen del backend y arranca los tres servicios (`db` → `migrate` → `backend`). La **primera vez** descarga PostgreSQL y las dependencias de Python: puede tardar varios minutos (por eso conviene hacerlo antes de la sesión). Las siguientes veces arranca en segundos.

Instrucciones

1. Confirmar que Docker está corriendo (en Windows/Mac, abrir Docker Desktop y esperar a que diga *running*):

```
docker --version
docker compose version
```

2. En la **terminal A** (carpeta `exploraec-curso/proyecto-curso-spec-kit`), levantar todo. **Deja esta terminal abierta**: aquí verás los registros del servidor (durante la práctica los usarás para ver las peticiones de la app):

```
cd exploraec-curso/proyecto-curso-spec-kit   # solo si aún no estás dentro del backend
docker compose up --build
```

3. Esperar a que aparezcan, en este orden, estas señales (los identificadores y tiempos varían):

```
 Container gastos-db Healthy
 Container gastos-migrate Started
gastos-migrate  | INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
gastos-migrate  | INFO  [alembic.runtime.migration] Will assume transactional DDL.
gastos-migrate  | INFO  [alembic.runtime.migration] Running upgrade  -> 0ae4f95a815b,
crear tablas usuarios y gastos
gastos-migrate  | INFO  [alembic.runtime.migration] Running upgrade 0ae4f95a815b ->
7c1d2e9a4b30, invalidar hashes bcrypt
gastos-migrate  | Hashes bcrypt invalidados: 0 usuario(s) requieren reset de
contraseña.
gastos-migrate  | INFO  [alembic.runtime.migration] Running upgrade 7c1d2e9a4b30 ->
a3f5c8d1e2b4, gastos: fecha, FK con ON DELETE CASCADE, indice por usuario y emails en
minusculas
 Container gastos-migrate Exited
 Container gastos-backend Started
gastos-backend  | INFO:     Application startup complete.
gastos-backend  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to
quit)
```

(En una terminal interactiva, Docker antepone un ✔ a las líneas `Container ...`; los identificadores y tiempos varían, y en la primera construcción verás antes muchas líneas de descarga e instalación.) `gastos-migrate` termina («Exited») **a propósito**: es el servicio de un solo uso que crea las tablas. Lo que debe quedar corriendo son `gastos-db` y `gastos-backend`.

4. Abrir **otra terminal** (una tercera, también en `exploraec-curso/proyecto-curso-spec-kit`, porque `docker compose ps` solo funciona dentro de la carpeta del backend) y comprobar los contenedores:

```
cd exploraec-curso/proyecto-curso-spec-kit
docker compose ps
```

5. **Opción B — backend del docente.** Si no puedes usar Docker, omite los pasos 1-4 y pide al docente su dirección en la red local (`http://<IP-del-docente>:8000`). Debes estar en la misma red Wi-Fi y usar esa dirección en lugar de `localhost` en el Paso 4 y, en el Paso 5, con `--dart-define`. Aun así haz el Paso 4.

Resultado esperado

`docker compose ps` muestra `gastos-db` y `gastos-backend` en ejecución (la base de datos como `healthy`) y el puerto `0.0.0.0:8000->8000/tcp`; el contenedor `gastos-migrate` no aparece (ya terminó). Algo parecido a (se acortó la columna `COMMAND`):

```
NAME             IMAGE                COMMAND   SERVICE   CREATED          STATUS
                    PORTS
gastos-backend   backend-backend      "uv ..."  backend   36 seconds ago   Up 28
seconds             0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
gastos-db        postgres:16-alpine   "dock..." db        36 seconds ago   Up 36
seconds (healthy)   5432/tcp
```

Comprobación en la base de datos. Comprueba que Postgres creó las tablas (terminal A, en `exploraec-curso/proyecto-curso-spec-kit`). Este es el comando que usarás en toda la práctica para **consultar la base de datos**: `docker compose exec db psql -U gastos -d gastos -c "<consulta SQL>"`.

```
docker compose exec db psql -U gastos -d gastos -c "\dt"
```

Debe listar tres tablas: `alembic_version`, `gastos` y `usuarios`.

Errores comunes

- **`Cannot connect to the Docker daemon`** / **`error during connect`**. *Solución:* Docker no está corriendo. Abre Docker Desktop (o `sudo systemctl start docker` en Linux) y repite el comando.

- **`permission denied while trying to connect to the Docker daemon socket` (Linux)**. *Solución:* usa `sudo docker compose ...` o agrega tu usuario al grupo `docker` (`sudo usermod -aG docker $USER` y volver a iniciar sesión).
- **`port is already allocated` / `address already in use` en el puerto 8000**. *Solución:* otro programa usa el 8000 (otro backend o un servidor local). Ciérralo, o cambia el mapeo en `docker-compose.yml` (`"8001:8000"`) y usa `API_BASE_URL` con ese puerto.
- **`gastos-backend` se detiene al arrancar** (aparece `Exited` o reinicia en bucle). *Solución:* `docker compose logs backend`. Casi siempre falta `SECRET_KEY` en `.env` (Paso 2).
- **`gastos-migrate` termina con error (`Exited (1)`) y el backend nunca arranca**. *Solución:* `docker compose logs migrate`. Si venías de un intento fallido con datos viejos, reinicia la base: `docker compose down -v` y vuelve a ejecutar `docker compose up --build` (esto **borra** los datos del backend).
- **Descarga muy lenta o se corta**. *Solución:* la primera vez bajan imágenes de varios cientos de MB; si falla por la red, repite el mismo comando: reanuda lo ya descargado.
- **Detener el backend al terminar**: `Ctrl+C` en la terminal de los registros, o `docker compose stop` desde otra terminal. La próxima vez basta `docker compose up` (sin `--build`).

✅ Antes de continuar, confirma que...

- ☐ Viste «Uvicorn running on http://0.0.0.0:8000» y «Application startup complete».
- ☐ `docker compose ps` muestra `gastos-db` y `gastos-backend` en ejecución (o tienes la dirección del backend del docente).

Paso 4 — Verificar el backend: `/health`, `/docs` y registro/login con `curl` (8 min)

Por qué se hace

Antes de escribir código en Flutter conviene comprobar que **el servidor responde y que entiendes lo que va a hacer la app**. Si algo falla después, sabrás que el problema está en la app y no en el backend. Y al repetir a mano el registro y el login verás exactamente la secuencia que `GastosApiService` automatiza en el Paso 7.

Instrucciones

1. **Salud del servidor.** En el navegador abre `http://localhost:8000/health` (o, desde la terminal, `curl -i http://localhost:8000/health`).

2. **Documentación interactiva.** Abre `http://localhost:8000/docs` (Swagger). Ubica los grupos **usuarios** (`POST /usuarios/` , `POST /usuarios/token`) y **gastos**. Déjala abierta: es tu referencia del contrato.
3. **Registro a mano** (usa un correo de práctica, distinto del que usarás en la app):

```
curl -X POST http://localhost:8000/usuarios/ \
  -H "Content-Type: application/json" \
  -d '{"email":"prueba@ejemplo.com","password":"clave-1234"}'
```

(En PowerShell, `curl` es un alias de otro comando: usa `curl.exe` ; en Git Bash funciona tal cual.)

4. **Login a mano.** Fíjate que se envía como **formulario** (`-d` con `campo=valor`), no como JSON, y que el campo se llama `username` aunque lleve el correo:

```
curl -X POST http://localhost:8000/usuarios/token \
  -d "username=prueba@ejemplo.com&password=clave-1234"
```

5. **Ruta protegida sin token.** Intenta listar gastos sin identificarte:

```
curl -i http://localhost:8000/gastos/
```

6. Mira la terminal donde corre `docker compose` : cada petición queda registrada.

Resultado esperado

1. `/health` responde `200` con:

```
{"status":"ok","db":"ok"}
```

(`503` con `"status":"degraded"` significaría que el servidor corre pero no alcanza la base de datos.)

2. `/docs` muestra la lista de endpoints agrupados.
3. El registro responde `201` con tu usuario (el correo se guarda en minúsculas):

```
{"id":1,"email":"prueba@ejemplo.com"}
```

4. El login responde `200` con el token (una cadena larga con tres partes separadas por puntos):

```
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"bearer"}
```

5. Sin token, `/gastos/` responde `401` con `{"detail":"Not authenticated"}`.
6. En los registros del backend aparecen líneas del estilo `POST /usuarios/ -> 201`, `POST /usuarios/token -> 200`, `GET /gastos/ -> 401`.
7. **Comprobación en la base de datos.** Comprueba que el usuario quedó guardado y que **la contraseña no se guarda en claro**, solo un hash (`left(...)` muestra sus primeros 10 caracteres). Terminal A:

```
docker compose exec db psql -U gastos -d gastos -c "SELECT id, email,
left(hashed_password, 10) AS inicio_del_hash FROM usuarios ORDER BY id;"
```

Debe mostrar una fila con `prueba@ejemplo.com` y un hash que empieza por `$argon2id$` (si ya habías registrado usuarios antes, verás más filas).

Errores comunes

- **`curl: (7) Failed to connect to localhost port 8000`**. *Solución:* el backend no está corriendo o aún está arrancando. Vuelve al Paso 3 (`docker compose ps`).
- **`{"detail":"El email prueba@ejemplo.com ya está registrado"}` (400)**. *Solución:* no es un fallo: ya lo registraste en otro intento. Pasa directo al login.
- **`{"detail":"Email o contraseña incorrectos"}` (401) en el login**. *Solución:* la contraseña no es la misma con la que registraste ese correo.
- **`422` con `"loc":["body","password"]` al registrar**. *Solución:* la contraseña tiene menos de 8 caracteres; el servidor exige de 8 a 72.
- **`422` con «Field required» y `username` en el login**. *Solución:* enviaste JSON; el login es un formulario (`-d "username=...&password=..."`, sin `-H "Content-Type: application/json"`).
- **`/health` responde `503`**. *Solución:* `docker compose ps`: ¿`gastos-db` está `healthy`? Si no, `docker compose logs db`.
- **Escribiste `/gastos` sin la barra final y ves `307 Temporary Redirect`**. *Solución:* no es un error: el servidor redirige a `/gastos/`. En `curl` usa la ruta con barra final (o `-L`).
- **Desde el emulador Android `localhost` no funciona** (se verá en el Paso 5): dentro del emulador, `localhost` es el propio emulador. Aquí, en tu computador, sí usas `localhost`.

✅ Antes de continuar, confirma que...

- ☐ `/health` responde `{"status":"ok","db":"ok"}` y `/docs` abre.
- ☐ Registraste un usuario y obtuviste un `access_token` con `curl`.
- ☐ Viste el `401` al pedir `/gastos/` sin token.

Paso 5 — Traer el código de la Sesión 6 y ejecutar ExploraEC (5 min)

Por qué se hace

El backend ya funciona; ahora la app necesita el código para hablar con él. La app lo alcanza por una dirección que **depende de dónde corre la app** (`localhost` dentro del emulador Android es el propio emulador, no tu computador).

Instrucciones

1. En tu proyecto `exploraec` (la **terminal B**, **no** la del backend), traer el código de la Sesión 6. Como en las sesiones anteriores, solo se traen los archivos que gestiona el repo `starter`:

```
cd exploraec-curso/exploraec   # solo si aún no estás dentro de tu proyecto
git fetch starter
git checkout starter/sesion-06 -- lib pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

Esto trae el servicio, el modelo, el controller y la pantalla de Gastos (archivos nuevos en `lib/`), la cuarta pestaña en `main.dart` y el `pubspec.yaml` con `http: ^1.2.2` ya agregado. **No** trae `android/` : esa carpeta no vive en el repo `starter` (la generó `flutter create` en tu proyecto), así que el manifiesto se edita a mano en el siguiente punto.

2. Abrir `android/app/src/main/AndroidManifest.xml` y hacer **dos cambios** (el archivo ya tiene los dos permisos de ubicación que agregaste en la Sesión 5):

- Agregar el permiso de internet justo debajo de los permisos de ubicación, como hijo directo de `<manifest>` .
- Agregar `android:usesCleartextTraffic="true"` como primer atributo de `<application` . Es solo para desarrollo: tu backend local usa `http://` , no `https://` , y Android 9+

bloquea `http://` por defecto. En producción esto se resuelve con HTTPS, no con este atributo.

Debe quedar así (lo nuevo está marcado con `<-- NUEVA`; **no copies** esas marcas al archivo):

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.INTERNET" />   <-- NUEVA
    <application
        android:usesCleartextTraffic="true"                          <-- NUEVA
        android:label="exploraec"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
```

(El resto del archivo — `<activity ...>`, `<meta-data ...>`, `<queries>` — no se toca. Si tu `<manifest` está en varias líneas, da igual: lo importante es que el permiso quede dentro de `<manifest>` y antes de `<application`.)

Comprueba que quedaron las dos líneas nuevas (desde la terminal B, en `exploraec-curso/exploraec`):

```
grep -n "INTERNET\|usesCleartextTraffic" android/app/src/main/AndroidManifest.xml
```

(En PowerShell: `Select-String "INTERNET|usesCleartextTraffic" android\app\src\main\AndroidManifest.xml`.) Debe mostrar exactamente dos líneas: la del permiso `INTERNET` y la de `android:usesCleartextTraffic="true"`.

3. Ejecutar la app. La dirección del backend se define con `--dart-define`; **en el emulador Android no hace falta nada** (el valor por defecto es `http://10.0.2.2:8000`, que es cómo el emulador ve el `localhost` de tu computador):

```
flutter run
```

Dónde corre la app	Cómo se ejecuta	`API_BASE_URL`
Emulador	`flutter run`	`http://10.0.2.2:8000` (por

Dónde corre la app	Cómo se ejecuta	`API_BASE_URL`
Android		defecto)
Simulador iOS	`flutter run --dart-define=API_BASE_URL=http://localhost:8000`	`http://localhost:8000`
Dispositivo físico u Opción B	`flutter run --dart-define=API_BASE_URL=http://<IP>:8000`	`http://<IP-de-la-máquina-con-el-backend>:8000` (misma red Wi-Fi)

Resultado esperado

La app corre con una cuarta pestaña, **Gastos**, que muestra un formulario de correo y contraseña con la dirección del backend en letra pequeña (`http://10.0.2.2:8000` en el emulador Android). Todavía no funciona al tocar «Entrar»: con cualquier correo y contraseña aparece en rojo «Pendiente: conecta el flujo en el Paso 7» (se conecta en ese paso). Inicio, Mapa y Favoritos siguen como al final de la Sesión 5; la barra inferior ahora tiene cuatro pestañas: Inicio, Mapa, Favoritos y Gastos. La primera ejecución compila el proyecto de nuevo y tarda más que un hot reload.

Comprobación de lo traído. En la terminal B, en `exploraec-curso/exploraec` :

```
git status --short
grep -n "http:" pubspec.yaml
```

(En PowerShell: `Select-String "http:" pubspec.yaml` .) `git status --short` debe listar, entre otros, `lib/config/api_config.dart` , `lib/controllers/gastos_controller.dart` , `lib/models/gasto.dart` , `lib/screens/gastos_screen.dart` , `lib/services/api_exception.dart` y `lib/services/gastos_api_service.dart` ; el `grep` debe mostrar `http: ^1.2.2` .

Errores comunes

- **`error: ruta especificada 'android/app/src/main/AndroidManifest.xml' no concordó con ningún archivo conocido por git`** (en inglés: `pathspec 'android/...' did not match any file(s) known to git`). *Solución:* el `checkout` no debe incluir `android/` : usa exactamente `git checkout starter/sesion-06 -- lib pubspec.yaml PLACEHOLDERS.md` y edita el manifiesto a mano (punto 2). Si el comando falló, no se trajo **nada**: repítelo corregido.
- **`Target of URI doesn't exist: 'package:http/http.dart'`** . *Solución:* confirmar que el

`pubspec.yaml` traído incluye `http: ^1.2.2` en `dependencies` y correr `flutter pub get` de nuevo.

- **`fatal: 'starter' does not appear to be a git repository`**. *Solución:* el remoto `starter` no está configurado en tu proyecto; pide al docente la URL y agrégalo con `git remote add starter <URL>`.
- **`flutter run` no refleja el `AndroidManifest.xml` editado**. *Solución:* los cambios de manifiesto no se aplican con hot reload: detén la app por completo y vuelve a ejecutar `flutter run`.
- **`The element type "uses-permission" must be terminated` / error de compilación en el manifiesto**. *Solución:* quedó una marca `<-- NUEVA` copiada al archivo o el permiso quedó fuera de `<manifest>`. Deja exactamente las dos líneas nuevas del bloque de arriba, sin las marcas.
- **La letra pequeña muestra una dirección distinta de la esperada**. *Solución:* quedó un `--dart-define` de un intento anterior; ejecuta `flutter run` sin flags (emulador Android).

✅ Antes de continuar, confirma que...

- ☐ La app compila y corre tras traer el código de la rama `sesion-06`, y ves la pestaña Gastos.
- ☐ El backend sigue corriendo en su terminal (`/health` responde).

Paso 6 — Revisar `GastosApiService` (7 min)

Por qué se hace

Antes de conectar la pantalla, conviene entender qué hace el servicio que ya viene completo: cómo se identifica ante el servidor, cómo se envía cada petición (no todas igual) y cómo traduce cada tipo de fallo a un mensaje que el usuario entienda.

Instrucciones

1. Abrir `lib/config/api_config.dart`: ubicar `API_BASE_URL` (viene de `--dart-define`, por defecto `http://10.0.2.2:8000`).
2. Abrir `lib/services/gastos_api_service.dart` y leerlo de principio a fin. Ubicar `registrar`: envía **JSON** (`jsonEncode` y la cabecera `Content-Type: application/json`).
3. Ubicar `login`. Es la petición «rara»: el endpoint `/usuarios/token` **no acepta JSON**, sino `application/x-www-form-urlencoded`, con el campo llamado `username` (que lleva tu

correo) y `password`:

```
final respuesta = await _enviar(() => http.post(
      Uri.parse('${ApiConfig.baseUrl}/usuarios/token'),
      // Un Map como body => `http` usa form-urlencoded por ti.
      body: {'username': email, 'password': password},
    ));
```

Al recibir `access_token`, el servicio lo guarda en `_token`: **solo en memoria**. No se escribe en disco ni en los registros (`print`); al cerrar la app se pierde y hay que iniciar sesión de nuevo. Guardarlo de forma segura es parte de la Sesión 8.

4. Ubicar `listarGastos`: envía `Authorization: Bearer <token>` y los parámetros `skip` y `limit` (paginación). El total de gastos que hay en el servidor **no** viene en el cuerpo, sino en la cabecera `X-Total-Count`.
5. Ubicar `_enviar`: es el único lugar con `try`/`catch` y `.timeout(...)`. Identificar qué mensaje corresponde a cada caso:

```
on SocketException      -> 'No hay conexión con el servidor...'
on http.ClientException -> 'No hay conexión con el servidor...' (misma causa; `http` a
 veces envuelve así el fallo)
on TimeoutException     -> 'El servidor tardó demasiado en responder...'
401                  -> 'Tu sesión caducó o las credenciales no son válidas...'
400 / 403 / 404      -> el texto de `detail` que manda el servidor
422                  -> `detail` llega como una LISTA de objetos {loc, msg, type}
5xx                  -> 'El servidor tuvo un problema. Inténtalo más tarde.'
```

6. Abrir `lib/models/gasto.dart` y ubicar `Gasto.fromJson`: identificar cómo se maneja un campo faltante o con otro tipo (`monto` puede llegar como entero o decimal).

Resultado esperado

Se puede explicar, en una frase cada una: por qué el login se envía distinto que el registro, dónde vive el token y por qué eso es provisional, y qué hace `_enviar` con un error `422` frente a uno `400`.

Errores comunes

- Ninguno específico de este paso: es de lectura, no de escritura de código.

✅ Antes de continuar, confirma que...

- ☐ Puedes explicar en una frase qué hace `GastosApiService.login` y por qué no usa JSON.
- ☐ Identificaste dónde se guarda el token y de dónde sale el total de gastos (`X-Total-Count`).

Paso 7 — Conectar `GastosController`: registro → login → listar gastos (10 min)

Por qué se hace

Este es el cambio central de la sesión: tres llamadas que **dependen una de otra** (no se puede listar sin token, ni pedir token sin cuenta). `await` permite escribirlas una tras otra como si fueran código normal. La interfaz de estados de la Sesión 3 y el patrón de controller de la Sesión 4 se mantienen: el controller guarda el estado, `Obx` reconstruye la pantalla.

Instrucciones

1. **Ver el estado inicial.** Con la app corriendo (Paso 5), en la pestaña Gastos escribe cualquier correo con una contraseña de 8 caracteres o más y toca «Entrar»: aparece en rojo «Pendiente: conecta el flujo en el Paso 7». Eso es lo que hacen hoy las líneas provisionales; ahora las reemplazas.
2. Abrir `lib/controllers/gastos_controller.dart` y ubicar el método `entrar(...)` con el marcador `TODO(sesion-06)`. Así lo trae la rama:

```
 Future<void> entrar(String email, String password) async {
   autenticando.value = true;
   mensajeAuth.value = '';

   // TODO(sesion-06): borra las dos líneas de abajo y descomenta el bloque completo.
(Paso 7 — flujo asíncrono)
   // Por qué: registro -> login -> listar dependen uno del otro; `await`
   // permite escribirlos en orden. Un 400 en el registro significa "el
   // correo ya existe": se ignora y se sigue al login.
   mensajeAuth.value = 'Pendiente: conecta el flujo en el Paso 7';
   autenticando.value = false;
   // try {
   //   try {
   //     await _api.registrar(email, password);
   //   } on ApiException catch (e) {
   //     if (e.statusCode != 400) rethrow;
   //   }
   //   await _api.login(email, password);
   //   sesionActiva.value = true;
   //   await cargarGastos();
   // } on ApiException catch (e) {
   //   mensajeAuth.value = e.mensaje;
   // } catch (e) {
   //   mensajeAuth.value = 'Error inesperado: $e';
   // } finally {
   //   autenticando.value = false;
   // }
 }
```

Haz **dos cambios**, y nada más (el `TODO` y los comentarios `// Por qué:` se quedan):

- **Borrar** las dos líneas provisionales: `mensajeAuth.value = 'Pendiente: conecta el flujo en el Paso 7';` y `autenticando.value = false;` (la que está justo encima de `// try {`). La que está dentro del `finally` **no** se toca.
- **Descomentar** el bloque: quitar el `//` del inicio de cada línea desde `// try {` hasta la `// }` que lo cierra. En VS Code: selecciona solo esas líneas y pulsa `Ctrl+/` (`Cmd+/` en Mac).

Debe quedar así:

```
 Future<void> entrar(String email, String password) async {
   autenticando.value = true;
   mensajeAuth.value = '';

   // TODO(sesion-06): borra las dos líneas de abajo y descomenta el bloque completo.
(Paso 7 — flujo asíncrono)
   // Por qué: registro -> login -> listar dependen uno del otro; `await`
   // permite escribirlos en orden. Un 400 en el registro significa "el
   // correo ya existe": se ignora y se sigue al login.
   try {
     try {
       await _api.registrar(email, password);
     } on ApiException catch (e) {
       if (e.statusCode != 400) rethrow;
     }
     await _api.login(email, password);
     sesionActiva.value = true;
     await cargarGastos();
   } on ApiException catch (e) {
     mensajeAuth.value = e.mensaje;
   } catch (e) {
     mensajeAuth.value = 'Error inesperado: $e';
   } finally {
     autenticando.value = false;
   }
 }
```

3. En el mismo archivo, ubicar `cargarGastos()` (también con `TODO(sesion-06)`). Así la trae la rama:

```
 Future<void> cargarGastos() async {
   estado.value = EstadoCarga.cargando;

   // TODO(sesion-06): borra las dos líneas de abajo y descomenta el bloque completo.
(Paso 7 — listar gastos)
   // Por qué: las 2 líneas fuerzan éxito con lista vacía, sin llamar a nada.
   // El bloque real traduce el resultado -o la excepción- a los mismos 3
   // estados de siempre; un 401 devuelve al formulario.
   gastos.value = [];
   estado.value = EstadoCarga.exito;
   // try {
   //   final resultado = await _api.listarGastos(skip: 0, limit: _tamanoPagina);
   //   gastos.value = resultado.gastos;
   //   totalEnServidor.value = resultado.total;
   //   estado.value = EstadoCarga.exito;
   // } on ApiException catch (e) {
   //   if (e.statusCode == 401) {
   //     sesionActiva.value = false; // volver al formulario
   //     mensajeAuth.value = e.mensaje; // y explicar por qué
   //   }
   //   mensajeError.value = e.mensaje;
   //   estado.value = EstadoCarga.error;
   // } catch (e) {
   //   mensajeError.value = 'Error inesperado: $e';
   //   estado.value = EstadoCarga.error;
   // }
 }
```

Mismos dos cambios: **borrar** `gastos.value = [];` y `estado.value = EstadoCarga.exito;` (las dos líneas que están justo encima de `// try {`; la que está dentro del bloque, después de `totalEnServidor.value = resultado.total;`, **no** se toca) y **descomentar** desde `// try {` hasta su `// }` final. Debe quedar así:

```
 Future<void> cargarGastos() async {
   estado.value = EstadoCarga.cargando;

   // TODO(sesion-06): borra las dos líneas de abajo y descomenta el bloque completo.
(Paso 7 — listar gastos)
   // Por qué: las 2 líneas fuerzan éxito con lista vacía, sin llamar a nada.
   // El bloque real traduce el resultado -o la excepción- a los mismos 3
   // estados de siempre; un 401 devuelve al formulario.
   try {
     final resultado = await _api.listarGastos(skip: 0, limit: _tamanoPagina);
     gastos.value = resultado.gastos;
     totalEnServidor.value = resultado.total;
     estado.value = EstadoCarga.exito;
   } on ApiException catch (e) {
     if (e.statusCode == 401) {
       sesionActiva.value = false; // volver al formulario
       mensajeAuth.value = e.mensaje; // y explicar por qué
     }
     mensajeError.value = e.mensaje;
     estado.value = EstadoCarga.error;
   } catch (e) {
     mensajeError.value = 'Error inesperado: $e';
     estado.value = EstadoCarga.error;
   }
 }
```

4. Ejecuta `flutter analyze` en la terminal B: debe terminar con `No issues found!`. Luego guarda y haz hot reload (en la terminal donde corre `flutter run`, pulsa `r`; en VS Code, guardar con `Ctrl+S` también recarga).
5. En la pestaña Gastos, escribir un correo tuyo (por ejemplo `tunombre@ejemplo.com`) y una contraseña de **8 a 72 caracteres**, y tocar «Entrar». (Si el mensaje en rojo «Pendiente…» sigue en pantalla, no pasa nada: se borra al tocar «Entrar».) La primera vez se registra la cuenta y se inicia sesión; las siguientes, el registro responde «ya existe» y se pasa directo al login.
6. Observar la secuencia: botón con indicador de espera → pantalla de carga → como tu usuario es nuevo y no tiene gastos, aparece el estado **vacío**, con el título `Gastos (0)`. (Los gastos se crean en el siguiente paso.)

Resultado esperado

Con credenciales válidas, la pantalla pasa del formulario a la pestaña de Gastos mostrando `EmptyView` («Aún no registras gastos en este viaje»). En la terminal A, donde corre `docker compose`, se ven las peticiones en este orden (la primera vez):

```
INFO     POST /usuarios/ -> 201 (68.3 ms)
INFO     POST /usuarios/token -> 200 (63.9 ms)
INFO     GET /gastos/ -> 200 (3.1 ms)
```

(Las siguientes veces el primer renglón es `POST /usuarios/ -> 400`: el correo ya existe, y la app lo ignora a propósito.)

Comprobación en la base de datos. En la terminal A comprueba que el servidor guardó tu cuenta y que aún no tienes gastos (cambia `tunombre@ejemplo.com` por tu correo):

```
docker compose exec db psql -U gastos -d gastos -c "SELECT id, email FROM usuarios
WHERE email = 'tunombre@ejemplo.com';"
docker compose exec db psql -U gastos -d gastos -c "SELECT count(*) AS total FROM
gastos g JOIN usuarios u ON u.id = g.usuario_id WHERE u.email =
'tunombre@ejemplo.com';"
docker compose logs --no-log-prefix backend | grep -E "INFO +(POST|GET)" | tail -3
```

(En PowerShell cambia el último comando por `docker compose logs --no-log-prefix backend | Select-String "INFO +(POST|GET)" | Select-Object -Last 3`.) La primera consulta devuelve una fila con tu correo; la segunda, `total` = 0, igual que el estado vacío `Gastos (0)` de la app; el último muestra las peticiones que hizo la app.

Errores comunes

- **«No hay conexión con el servidor» (en el emulador Android)**. *Solución:* revisar que el backend esté corriendo (`/health` en el navegador de tu computador) y que no pasaste un `API_BASE_URL` con `localhost`: dentro del emulador, `localhost` es el propio emulador; el computador se ve como `10.0.2.2`. En simulador iOS sí se usa `localhost`; en un dispositivo físico, la IP de la red local.
- **`Cleartext HTTP traffic to 10.0.2.2 not permitted`**. *Solución:* Android 9+ bloquea `http://` por defecto. Confirmar que el `AndroidManifest.xml` que editaste en el Paso 5 contiene `android:usesCleartextTraffic="true"` dentro de `<application`, y **detener y volver a ejecutar** `flutter run` (el manifiesto no se recarga en caliente). En producción esto se resuelve con HTTPS, no con este permiso.

- **Mensaje de «Datos inválidos» al registrarte**. *Solución:* la contraseña debe tener entre 8 y 72 caracteres y el correo debe tener un formato válido; el servidor responde `422` con el detalle de cada campo.
- **«Tu sesión caducó o las credenciales no son válidas»**. *Solución:* el correo ya estaba registrado con otra contraseña. Usa la contraseña con la que lo creaste, o registra otro correo.
- **El formulario no pasa a Gastos y no muestra ningún mensaje**. *Solución:* quedaron activas las líneas provisionales (`mensajeAuth.value = 'Pendiente...'`). Hay que borrarlas, no solo comentar el bloque nuevo.
- **`Null check operator used on a null value` al listar**. *Solución:* revisar que se trajo `gasto.dart` completo desde la rama `sesion-06` (`git checkout starter/sesion-06 -- lib`).

✅ Antes de continuar, confirma que...

- ☐ Te registraste e iniciaste sesión desde la app, y llegaste al estado vacío de Gastos.
- ☐ En la terminal del backend viste las tres peticiones en el orden esperado.

Paso 8 — Probar los 3 estados y los errores con el servidor real (8 min)

Por qué se hace

La interfaz de estados solo demuestra su valor cuando se prueba con condiciones reales. Aquí no hace falta un menú de simulación: el **vacío** es el de tu usuario nuevo, el **éxito** lo creas tú, y los **errores** los provocas a propósito en el servidor.

Instrucciones

1. **Éxito.** Crea 3 gastos para **tu usuario de la app** (el mismo correo y contraseña del Paso 7; los gastos son por usuario). Hay dos formas; elige una:

- **Swagger — abre `http://localhost:8000/docs`** (la página descarga Swagger UI desde internet; sin conexión se ve en blanco y hay que usar `curl`). Toca **Authorize** (arriba a la derecha). Se abre «Available authorizations» con los campos `username` , `password` , *Client credentials location*, `client_id` y `client_secret` .
- **Escribe tu correo en `username` y tu contraseña en `password`** (los de la app) y deja lo demás vacío. Toca el botón **Authorize** del diálogo: debe aparecer «Authorized» con tu

correo. Toca **Close**: los candados de los endpoints quedan cerrados y Swagger enviará tu token solo en cada petición.

- **Abre `POST /gastos/` y toca *Try it out*.** El cuerpo de ejemplo trae valores de relleno (`"descripcion": "string"`, `"monto": 1000000`, `"categoria": "string"` y una `fecha`); si lo ejecutas tal cual responde `400` «'string' no es una categoría válida». Selecciona **todo** el contenido del cuadro (`Ctrl+A`) y escribe:

```
{"descripcion": "Almuerzo en el Mercado Central", "monto": 6.50, "categoria":
"comida"}
```

- **Toca *Execute*.** Debajo aparece el `curl` equivalente (con tu `Authorization: Bearer ...`) y la respuesta del servidor: código `201` con el gasto creado. Repite dos veces más cambiando la descripción, el monto y la categoría (`comida`, `transporte`, `entretenimiento` u `otros`, en minúsculas). La `fecha` la pone el servidor con la fecha en hora UTC; en Ecuador, desde las 19:00 puede ser la del día siguiente.

- **curl** (Linux/Mac/Git Bash; en la terminal A o cualquier otra). Nota que el login se envía como formulario, igual que en la app. Cambia el correo y la contraseña por los tuyos:

```
TOKEN=$(curl -s -X POST http://localhost:8000/usuarios/token \
  -d "username=tunombre@ejemplo.com&password=tu-clave-1234" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")
curl -X POST http://localhost:8000/gastos/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"descripcion":"Almuerzo en el Mercado
Central","monto":6.50,"categoria":"comida"}'
curl -X POST http://localhost:8000/gastos/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"descripcion":"Taxi al terminal","monto":3.25,"categoria":"transporte"}'
curl -X POST http://localhost:8000/gastos/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"descripcion":"Entrada al Museo Casa del
Alabado","monto":4,"categoria":"entretenimiento"}'
```

Cada `curl` responde `201` con el gasto creado, por ejemplo `{"id":1,"descripcion":"Almuerzo en el Mercado Central","monto":6.5,"categoria":"comida","fecha":"2026-10-05"}`.

2. De vuelta en la app, tocar el ícono de recargar de la barra superior: el estado pasa por

cargando y muestra los 3 gastos (descripción, `categoría · fecha` y el monto a la derecha, con dos decimales), con el título `Gastos (3)` tomado de `X-Total-Count`.

3. **Error de conexión (SocketException).** Detener el backend desde otra terminal, dentro de la carpeta del backend:

```
cd exploraec-curso/proyecto-curso-spec-kit
docker compose stop backend
```

Recargar en la app: debe aparecer `ErrorView` con «No hay conexión con el servidor. Revisa tu red y que el backend esté encendido.», no un error técnico crudo ni una app congelada. Volver a levantarlo y, cuando `http://localhost:8000/health` responda, tocar «Reintentar» (la lista vuelve a cargar):

```
docker compose start backend
```

(Con un dispositivo físico también puedes activar el modo avión.)

4. **Error 401.** En el menú « ⋮ » de Gastos, elegir «Invalidar token (solo práctica)» y luego tocar el ícono de recargar: el servidor rechaza el token y la app vuelve al formulario mostrando en rojo «Tu sesión caducó o las credenciales no son válidas. Inicia sesión de nuevo.». Escribe de nuevo tu correo y tu contraseña (el formulario queda vacío) y toca «Entrar»: vuelven los 3 gastos.
5. **Error 422.** Ya estás en el formulario (si no, menú « ⋮ » → «Cerrar sesión»). Escribe un correo **nuevo** (por ejemplo `nuevo@ejemplo.com`) y una contraseña de 3 caracteres (`abc`) y toca «Entrar»: debe aparecer en rojo «Datos inválidos — password: String should have at least 8 characters», con el campo señalado y no el JSON en bruto. (El texto después de los dos puntos viene del servidor y está en inglés: es esperado.)
6. **Credenciales incorrectas (opcional).** Escribe un correo que ya registraste con una contraseña distinta de la que usaste (por ejemplo, `clave-0000`): el registro responde «ya existe» (la app lo ignora) y el login responde 401, así que aparece «Tu sesión caducó o las credenciales no son válidas. Inicia sesión de nuevo.».

Resultado esperado

Los 3 estados (loading, vacío, error) se vieron con el servidor real, y cada error produjo un mensaje distinto y legible: sin conexión, sesión caducada y datos inválidos. Mientras se ve el error por falta de conexión, la lista previa **no se borra** del controller (solo cambia el estado), y eso es lo esperado.

Comprobación en la base de datos y en la API. En la terminal A comprueba que los 3

gastos están guardados y que coinciden con lo que muestra la app (cambia el correo y la contraseña por los tuyos):

```
docker compose exec db psql -U gastos -d gastos -c "SELECT g.id, g.descripcion,
g.monto, g.categoria, g.fecha FROM gastos g JOIN usuarios u ON u.id = g.usuario_id
WHERE u.email = 'tunombre@ejemplo.com' ORDER BY g.id;"
TOKEN=$(curl -s -X POST http://localhost:8000/usuarios/token -d
"username=tunombre@ejemplo.com&password=tu-clave-1234" | python3 -c "import sys,json;
print(json.load(sys.stdin)['access_token'])")
curl -si "http://localhost:8000/gastos/?limit=1" -H "Authorization: Bearer $TOKEN" |
grep -i "^HTTP\|x-total-count"
```

La consulta debe mostrar **3 filas** con las mismas descripciones, montos y categorías de la lista de la app; el `curl` debe mostrar `HTTP/1.1 200 OK` y `x-total-count: 3`, el número que la app muestra en `Gastos (3)`.

Para cada error que provocaste, esta es la comprobación de que el servidor se comportó como la app lo mostró:

```
docker compose ps
curl -i http://localhost:8000/gastos/ -H "Authorization: Bearer token-invalido"
docker compose exec db psql -U gastos -d gastos -c "SELECT id, email FROM usuarios
WHERE email = 'nuevo@ejemplo.com';"
```

- Con el backend detenido (`docker compose stop backend`), `docker compose ps` lista solo `gastos-db`; tras `docker compose start backend` aparece también `gastos-backend` como `Up`.
- El segundo comando devuelve `401` con `{"detail":"No se pudo validar las credenciales"}`: lo mismo que recibió la app con «Invalidar token».
- La consulta devuelve `(0 rows)`: el `422` por contraseña corta rechazó el registro **antes** de guardar nada (usa el correo con el que probaste en el punto 5).

Errores comunes

- **La app se congela en vez de mostrar el `ErrorView` al detener el backend**. *Solución:* revisar que `_enviar` en `GastosApiService` sigue intacto (no se haya sobrescrito al copiar archivos): sin su `try` / `catch`, la excepción no se traduce a un mensaje.
- **`POST /gastos/` responde `400` con «Este gasto supera el límite de 500.0…»**. *Solución:* el backend limita lo acumulado por categoría a 500. Usa montos pequeños; es

una regla de negocio que se trabaja en la Sesión 8.

- **`422` al crear el gasto en Swagger**. *Solución:* revisa que `monto` sea un número mayor que 0 y que `categoria` esté en minúsculas y sea una de las cuatro permitidas.
- **Tras reiniciar el backend con `docker compose up`, el estado vacío y los datos se ven distintos**. *Solución:* si borraste los volúmenes (`down -v`), la base se reinició y tu usuario ya no existe; la app lo vuelve a registrar al entrar.

✅ Antes de continuar, confirma que...

- ☐ Viste el estado vacío, el de éxito con tus 3 gastos y el de error por falta de conexión.
- ☐ Provocaste un 401 y un 422 y leíste mensajes comprensibles, no JSON en bruto.

Paso 9 — Aplicar la teoría: un timeout explícito (6 min)

Por qué se hace

La teoría de hoy insiste en que toda llamada de red lleva un límite de tiempo: sin él, una respuesta que nunca llega deja la pantalla cargando para siempre. `GastosApiService` ya lo tiene (15 segundos); aquí lo acortas a propósito para ver el camino del `TimeoutException` de punta a punta.

Instrucciones

1. En `lib/services/gastos_api_service.dart`, dentro de `_enviar`, ubicar `.timeout(const Duration(seconds: 15))` y cambiarlo por:

```
.timeout(const Duration(milliseconds: 1))
```

Comprueba el cambio (terminal B, en `exploraec-curso/exploraec`):

```
grep -n "timeout(" lib/services/gastos_api_service.dart
```

(En PowerShell: `Select-String "timeout\(" lib\services\gastos_api_service.dart`.) Debe mostrar una sola línea con `Duration(milliseconds: 1)`; al devolverlo debe mostrar `Duration(seconds: 15)`. 2. Guardar y hacer hot reload (con la sesión ya iniciada, para no probar el login). En Gastos tocar el ícono de recargar. 3. Observar el `ErrorView`: el mensaje debe ser «El servidor tardó demasiado en responder. Revisa tu conexión e inténtalo de nuevo.», no un error técnico. 4. Devolver el valor a `Duration(seconds: 15)`, guardar, tocar

«Reintentar» y confirmar que los gastos vuelven a cargar.

Resultado esperado

Con 1 ms la carga siempre falla con el mensaje de «tardó demasiado», traducido por el servicio y mostrado por el mismo `ErrorView` de la Sesión 3, aunque el backend esté en tu propio computador y responda bien; con 15 s vuelve a funcionar.

Errores comunes

- **Sigue cargando normalmente con el valor de 1 ms**. *Solución:* el cambio no se aplicó: confirmar que se editó el `.timeout(...)` de `_enviar` (el único del archivo) y que se hizo hot reload.
- **Quedó el timeout de 1 ms al terminar**. *Solución:* restaurar `Duration(seconds: 15)` antes de cerrar: es el valor que espera la Sesión 7.
- **Con 1 ms ni siquiera puedo iniciar sesión**. *Solución:* es lo esperado, porque `login` también pasa por `_enviar`. Haz la prueba con la sesión ya iniciada.

✅ Antes de continuar, confirma que...

- ☐ Con 1 ms viste el mensaje «El servidor tardó demasiado en responder…».
- ☐ Restauraste `Duration(seconds: 15)` y la lista volvió a cargar.

Paso 10 (opcional) — Deslizar para actualizar (10 min)

> Este paso **no cuenta dentro de los 65 minutos** ni bloquea la Sesión 7. Hazlo si terminaste antes que el resto o como repaso. Sigue la misma mecánica de los pasos anteriores: leer el `TODO`, borrar la línea indicada, descomentar el bloque y probarlo en la app.

Por qué se hace

Hasta ahora, para volver a pedir los gastos había que tocar el ícono de recargar. El gesto natural en un móvil es **deslizar la lista hacia abajo**. `RefreshIndicator` lo da hecho, pero su parámetro `onRefresh` exige una función que devuelva un `Future`: el indicador gira **hasta que ese `Future` termina**. Es la teoría de la sesión (`Future`, `async`/`await`) aplicada a un widget real. Como `cargarGastos` ya es `async`, se pasa tal cual, sin escribir nada nuevo. La lista ya viene preparada con `AlwaysScrollableScrollPhysics` para poder deslizarla aunque quepa entera en pantalla.

Instrucciones

1. En `lib/screens/gastos_screen.dart`, ubicar el `TODO(sesion-06): OPCIONAL` (dentro del `Obx` del `body`) y leer su comentario `// Por qué:`. Así lo trae la rama:

```
        // TODO(sesion-06): OPCIONAL — borra la línea de abajo y descomenta el bloque
completo. (Paso 10 — deslizar para actualizar)
        // Por qué: `RefreshIndicator.onRefresh` exige una función que devuelva
        // un `Future` — el indicador gira hasta que ese `Future` termina.
        // `cargarGastos` ya es `async`, así que se pasa tal cual: no hace falta
        // escribir nada nuevo. Mientras recarga, el `Obx` de arriba muestra el
        // `LoadingView` de siempre; eso es lo esperado.
        return _buildLista(controller.gastos);
        // return RefreshIndicator(
        //   onRefresh: controller.cargarGastos,
        //   child: _buildLista(controller.gastos),
        // );
```

2. **Borrar** la línea `return _buildLista(controller.gastos);` y **descomentar** el bloque `return RefreshIndicator(...)` que sigue (quitar el `//` de esas cuatro líneas). Debe quedar así:

```
        // TODO(sesion-06): OPCIONAL — borra la línea de abajo y descomenta el bloque
completo. (Paso 10 — deslizar para actualizar)
        // Por qué: `RefreshIndicator.onRefresh` exige una función que devuelva
        // un `Future` — el indicador gira hasta que ese `Future` termina.
        // `cargarGastos` ya es `async`, así que se pasa tal cual: no hace falta
        // escribir nada nuevo. Mientras recarga, el `Obx` de arriba muestra el
        // `LoadingView` de siempre; eso es lo esperado.
        return RefreshIndicator(
          onRefresh: controller.cargarGastos,
          child: _buildLista(controller.gastos),
        );
```

3. Guardar, hacer hot reload (`r`) y, en Gastos, deslizar la lista hacia abajo desde el primer elemento (en el emulador, arrastrar con el mouse).
4. Observar: aparece el indicador de carga, el `Obx` muestra el `LoadingView` y regresa la lista con los gastos recién consultados.

Resultado esperado

Deslizar hacia abajo vuelve a consultar los gastos (los mismos 3 estados de siempre). No cambió ningún otro archivo: la pantalla sigue leyendo el mismo `GastosController`.

Comprobación en el servidor. Cada vez que la app pide la lista, el backend registra un `GET /gastos/ -> 200`. En la terminal A ejecuta esto antes y después de deslizar la lista:

```
docker compose logs --no-log-prefix backend | grep -c "GET /gastos/ -> 200"
```

(En PowerShell: `(docker compose logs --no-log-prefix backend | Select-String "GET /gastos/ -> 200").Count`.) El número debe subir en 1 por cada vez que deslizas.

Errores comunes

- **Deslizar no hace nada**. *Solución:* quedaron las dos versiones; el primer `return` termina la función y el `RefreshIndicator` nunca se ejecuta (el analizador marca `dead_code`). Borrar `return _buildLista(controller.gastos);`, no solo comentarla.
- **Deslizar no hace nada en el estado «vacío» ni en el de error**. *Solución:* es lo esperado: en esos estados se muestran `EmptyView` y `ErrorView`, que no son una lista deslizable. Para reintentar en error está el botón «Reintentar».
- **Al deslizar se ve el `LoadingView` en vez del indicador circular de siempre**. *Solución:* es lo esperado: `cargarGastos` pone el estado en `cargando`, y el `Obx` reemplaza la lista por el `LoadingView` mientras dura la consulta.
- **Tras deslizar aparece «Tu sesión caducó…»**. *Solución:* el token dura 30 minutos; vuelve a entrar desde el formulario.

✅ Antes de continuar, confirma que...

- ☐ Deslizar la lista hacia abajo vuelve a consultar los gastos.
- ☐ Puedes explicar por qué `onRefresh` necesita un `Future` y por qué `cargarGastos` se puede pasar sin envolverla.

Mini-reto *(sin solución escrita)*

Avisar al terminar. Muestra un `Get.snackbar('Lista actualizada', ...)` cuando termine la recarga. Pista: `onRefresh` acepta una función `async`, así que puedes hacer `await controller.cargarGastos();` y poner el aviso después.

Anexo — Todos los comandos de la práctica, en orden

Resumen para copiar y pegar, en el mismo orden de los pasos (Linux/Mac; en Windows usa las variantes indicadas en cada paso). Las líneas `# [editar a mano]` son cambios de archivo que se explican en el paso indicado: no son comandos. `<id-del-emulador>` es el id que muestra `flutter emulators`.

Antes de empezar — Paso 0 (terminal B, en tu proyecto `exploraec`):

```
git status
git add -A && git commit -m "Sesión 5"        # solo si git status muestra cambios sin
 guardar
flutter emulators
flutter emulators --launch <id-del-emulador>
flutter devices
adb emu geo fix -78.5125 -0.2201              # Quito (longitud primero)
flutter run
adb emu geo fix -79.8891 -2.1894              # Guayaquil; en la app: Inicio y luego
Mapa
# q en la terminal de flutter run para detenerla
```

Pasos 2.1 a 2.5 — carpeta común, backend y `.env`:

```
cd ~/Documentos
mkdir exploraec-curso
mv exploraec exploraec-curso/exploraec
cd exploraec-curso/exploraec
flutter pub get
cd ..
git clone https://github.com/cj-murillo/proyecto-curso-spec-kit
cd proyecto-curso-spec-kit
cp .env.example .env
python3 -c "import re,secrets,pathlib; p=pathlib.Path('.env');
p.write_text(re.sub(r'(?m)^SECRET_KEY=.*', 'SECRET_KEY='+secrets.token_hex(32),
p.read_text()))"
grep SECRET_KEY .env
cd ..
code .
```

Paso 3 y 4 — terminal A (backend):

```
cd exploraec-curso/proyecto-curso-spec-kit
docker --version
docker compose version
docker compose up --build                    # deja esta terminal abierta
```

En **otra** terminal, también en `exploraec-curso/proyecto-curso-spec-kit`:

```
docker compose ps
curl -i http://localhost:8000/health
curl -X POST http://localhost:8000/usuarios/ -H "Content-Type: application/json" -d
'{"email":"prueba@ejemplo.com","password":"clave-1234"}'
curl -X POST http://localhost:8000/usuarios/token -d
"username=prueba@ejemplo.com&password=clave-1234"
curl -i http://localhost:8000/gastos/
```

Y en el navegador: `http://localhost:8000/docs`.

Paso 5 — terminal B (app):

```
cd exploraec-curso/exploraec
git fetch starter
git checkout starter/sesion-06 -- lib pubspec.yaml PLACEHOLDERS.md
flutter pub get
# [editar a mano] android/app/src/main/AndroidManifest.xml: permiso INTERNET +
android:usesCleartextTraffic="true" (Paso 5, punto 2)
grep -n "INTERNET\|usesCleartextTraffic" android/app/src/main/AndroidManifest.xml
flutter run
```

Paso 7:

```
# [editar a mano] lib/controllers/gastos_controller.dart: entrar() y cargarGastos()
(Paso 7, puntos 3 y 4)
flutter analyze
# r en la terminal de flutter run (hot reload)
```

Paso 8:

```
TOKEN=$(curl -s -X POST http://localhost:8000/usuarios/token -d
"username=tunombre@ejemplo.com&password=tu-clave-1234" | python3 -c "import sys,json;
print(json.load(sys.stdin)['access_token'])")
curl -X POST http://localhost:8000/gastos/ -H "Authorization: Bearer $TOKEN" -H
"Content-Type: application/json" -d '{"descripcion":"Almuerzo en el Mercado
Central","monto":6.50,"categoria":"comida"}'
curl -X POST http://localhost:8000/gastos/ -H "Authorization: Bearer $TOKEN" -H
"Content-Type: application/json" -d '{"descripcion":"Taxi al
terminal","monto":3.25,"categoria":"transporte"}'
curl -X POST http://localhost:8000/gastos/ -H "Authorization: Bearer $TOKEN" -H
"Content-Type: application/json" -d '{"descripcion":"Entrada al Museo Casa del
Alabado","monto":4,"categoria":"entretenimiento"}'
```

En la terminal A:

```
docker compose stop backend
docker compose start backend
```

Paso 9 (terminal B):

```
# [editar a mano] lib/services/gastos_api_service.dart: .timeout(const
Duration(milliseconds: 1))
grep -n "timeout(" lib/services/gastos_api_service.dart
# r en la terminal de flutter run
# [editar a mano] devolver el valor a Duration(seconds: 15)
grep -n "timeout(" lib/services/gastos_api_service.dart
# r en la terminal de flutter run
```

Paso 10 (opcional):

```
# [editar a mano] lib/screens/gastos_screen.dart: RefreshIndicator (Paso 10, puntos 1
y 2)
flutter analyze
# r en la terminal de flutter run
```

Comprobaciones (terminal A, en `exploraec-curso/proyecto-curso-spec-kit`): consultar la base de datos y los registros del servidor después de cada cambio. Cambia el correo por el

tuyo.

```
docker compose exec db psql -U gastos -d gastos -c "\dt"
docker compose exec db psql -U gastos -d gastos -c "SELECT id, email,
left(hashed_password, 10) AS inicio_del_hash FROM usuarios ORDER BY id;"
docker compose exec db psql -U gastos -d gastos -c "SELECT g.id, g.descripcion,
g.monto, g.categoria, g.fecha FROM gastos g JOIN usuarios u ON u.id = g.usuario_id
WHERE u.email = 'tunombre@ejemplo.com' ORDER BY g.id;"
docker compose logs --no-log-prefix backend | grep -E "INFO +(POST|GET)" | tail -3
curl -i http://localhost:8000/gastos/ -H "Authorization: Bearer token-invalido"
```

Al terminar (terminal A): `Ctrl+C` en la terminal de los registros o `docker compose stop`; la próxima vez basta `docker compose up`.

Entregable de la práctica

- Backend de gastos corriendo en tu computador con Docker (`/health` responde `{"status":"ok","db":"ok"}`), con un usuario registrado y un token obtenido a mano con `curl`.
- Sección Gastos funcionando en ExploraEC: registro, inicio de sesión (form-urlencoded) y lista de gastos desde el backend, con el token solo en memoria.
- Los 3 estados (loading/vacío/error) verificados con el servidor real, incluyendo sin conexión, 401 y 422.
- Un timeout explícito probado de punta a punta (Paso 9) y restaurado a 15 segundos.
- *(Opcional)* Deslizar para actualizar (Paso 10).
- Tu práctica subida según lo indicado en clase. Lo que se sube es el proyecto `exploraec`; la carpeta `exploraec-curso` solo agrupa el front y el back en tu computador (no es un repositorio) y el backend (`proyecto-curso-spec-kit`, con su `.env`) **no** se sube.

Conexión con la siguiente sesión

La **Sesión 7 empieza con el Quiz 1** (15 min, Sesiones 1 a 5): repasa tus notas y esta práctica. Después se agrega persistencia local con Hive: la lista de gastos se guarda como caché para poder consultarla sin conexión, y los favoritos de lugares sobreviven cerrar la app. Notarás que el token sigue **solo en memoria** (hay que iniciar sesión cada vez); guardarlo de forma segura y completar el CRUD de gastos llega en la Sesión 8. Es indispensable llegar con la Sesión 6 funcionando **y con el backend levantado** (`docker compose up`).